CLC TP3 Number 20250328

UDC 004 Available for reference □Yes □No

Southern University of Science and Technology

Undergraduate Thesis

Thesis Title: **ABIscope —Differential Analysis of Linux ABI Compatibility in Rust-based Kernels**

Student Name: **杨若谷**

Student ID: **12213043**

Department: **Computer Science and Engineering**

Program: **Computer Science and Technology**

Thesis Advisor: **张殷乾**

Apr 02, 2026

COMMITMENT OF HONESTY

1. I solemnly promise that the paper presented comes from my independent research work under my supervisor's supervision. All statistics and images are real and reliable.

2. Except for the annotated reference, the paper contents no other published work or achievement by person or group. All people making important contributions to the study of the paper have been indicated clearly in the paper.

3. I promise that I did not plagiarize other people's research achievement or forge related data in the process of designing topic and research content.

4. If there is violation of any intellectual property right, I will take legal responsibility myself.

Signature:

Date:

ABIscope —Differential Analysis of Linux ABI Compatibility in Rust-based Kernels

杨若谷

(Computer Science and Engineering Thesis Advisor: 张殷乾)

[Abstract]: Linux kernel is the core component of the Linux operating system, responsible for providing essential services and managing hardware resources. With the emergence of Rust as a systems programming language, there has been growing interest in using Rust for kernel development due to its memory safety features. However, ensuring ABI (Application Binary Interface) Compatibility between Rust-based kernels and existing Linux kernel modules is crucial for the successful adoption of Rust in kernel development. Existing tools like Syzkaller target at finding bugs in the Linux kernel, but they do not specifically address the challenges of ABI Compatibility in Rust-based kernels. In this thesis, we present ABIscope, a novel tool designed to generate user-space test cases for differential analysis of Linux ABI Compatibility in Rust-based kernels. ABIscope makes uses of LLMs to generate test cases. We evaluate ABIscope on Rust-based kernels and demonstrate its effectiveness in identifying ABI Compatibility issues, providing valuable insights for developers and researchers in the field of kernel development.

[Key Words]: Linux Kernel, Rust, ABI Compatibility, LLMs, Syzkaller

目录

1. Introduction **1**

2. Methodology **3**

2.1 Overview 3

2.2 Seed Initialization 4

2.3 LLM-based mutation 5

2.4 Execution and Validation 6

2.5 Coverage-Guided Selection 6

2.6 Threats to Validity 7

3. Initial Experimental Results **8**

3.1 KeepOS 8

3.2 Asterinas 8

4. Discussion **9**

5. Summary and Future Work **9**

References **10**

Appendix **11**

Appendix A Code Example 11

1. Introduction

Linux operating system has been widely adopted in various domains from personal computers to servers and embedded systems(references). The Linux kernel is the core component of the Linux operating system. It is essential for providing different kinds of crucial services like process management,file system and so on. Linux kernel is written mostly in C, which is a powerful but unsafe programming language because of its lack of memory safety features(references). These days, a memory safety issue in kernels can finally result in a security vulnerability which be attacked by the malicious users and result in a severe consequence(references). Rust is a modern programming language that provides memory safety guarantees without sacrificing performance. It has become more and more popular in the systems programming community. Researchers begin to use Rust to write kernel modules and even entire kernels like KeepOS, Asterinas, Kerla and so on(references).

However, many existing Linux kernel modules like device drivers and file systems are written in C. They can not directly work with Rust-based kernels. Based on this, ideas of using Rust to write Linux kernel modules is proposed.(references). However, ensuring ABI(Application Binary Interface) Compatibility between Rust-based kernels and existing Linux kernel modules is crucial for the successful adoption of Rust in kernel development. ABI Compatibility means that the binary interface between the kernel and user-space applications or kernel modules remains consistent, allowing them to interact seamlessly without requiring modifications. Ensuring ABI Compatibility is essential for maintaining the stability and functionality of existing software while enabling the benefits of Rust's memory safety features in kernel development.

Although kernel developers have made efforts to ensure ABI Compatibility when building Rust-based kernels, there are still challenges and potential issues thay can arise. For most rust-based kernels, developers just write in the documentation that the kernel they develop is ABI compatible with the Linux kernel, but they do not provide a systematic way to verify the ABI Compatibility. Existing tools like Syzkaller target at finding bugs in the Linux kernel, but these

tools cannot directly use in the Rust-based kernels because of the differences in the kernel implementation and the lack of support for Rust-based kernels. What's more, these tools target at finding bugs in the kernel, but finding bugs is not the same as finding ABI Compatibility issues. ABI Compatibility issues can be more subtle and may not be regarded as bugs in the kernel. In this case, there is a need for a systematic way to verify the ABI Compatibility of Rust-based kernels with existing Linux kernel modules.

Recently, large language models (LLMs) have shown great potential in various domains, including software development, code generation, and testing. Many LLM-based test generation methods have been proposed, demonstrating strong capabilities in generating high-quality inputs and exploring complex program behaviors. However, directly applying these techniques to kernel-level ABI compatibility testing remains challenging due to the complexity of kernel interfaces and the lack of well-defined specifications.

In particular, the Linux ABI is highly complex, involving numerous system calls, complicated data structures, and subtle interactions between user space and kernel space. Exhaustively verifying ABI compatibility across all possible interfaces is therefore infeasible in practice. To address this challenge, we propose to focus on a critical and well-defined subset of the ABI: system calls. System calls serve as the primary interface between user applications and the kernel, and thus provide an effective abstraction for testing ABI compatibility.

Based on this insight, we design and implement Abiscope, a seed generation tool tailored for ABI compatibility testing of Rust-based kernels. Abiscope leverages LLMs to generate system call sequences (seeds) and incorporates an intermediate parameter propagation mechanism, where arguments from previous system calls are reused and transformed for subsequent calls. This design enables Abiscope to construct semantically meaningful and stateful test cases, rather than isolated and random inputs.

Furthermore, Abiscope improves system call argument coverage by systematically exploring diverse parameter combinations and dependencies. By increasing the diversity and validity of system call inputs, Abiscope indirectly enhances kernel code coverage, as measured

by KCOV. This approach allows us to effectively exercise deeper execution paths in the kernel without requiring intrusive instrumentation or kernel-specific adaptations.

We evaluate Abiscope on the file system subsystem of multiple Rust-based kernels, including Asterinas and KeepOS and compare their behaviors against the Linux kernel. Our results reveal several previously undocumented ABI incompatibility issues, demonstrating that even kernels claiming ABI compatibility may exhibit inconsistencies in system call semantics and behaviors. These findings highlight the necessity of systematic ABI verification and showcase the effectiveness of Abiscope in uncovering such issues.

2. Methodology

2.1 Overview

We present ABIscope, a novel tool designed to generate user-space test cases for differential analysis of Linux ABI Compatibility in Rust-based kernels target at syscalls. The Overall pipeline follows an iterative seed generation and evaluation process, leveraging LLMs with feedback-driven exploration. At a high level, ABIscope consists of four key stages: (1) Seed initialization (Section 2.2), (2) LLM-based mutation (Section 2.3), (3) execution and validation (Section 2.4), and (4) coverage-guided selection (Section 2.5). These four key components form a closed feedback loop. The overall pipeline is illustrated in Figure 1. We describe each stage in detail in the following sections.

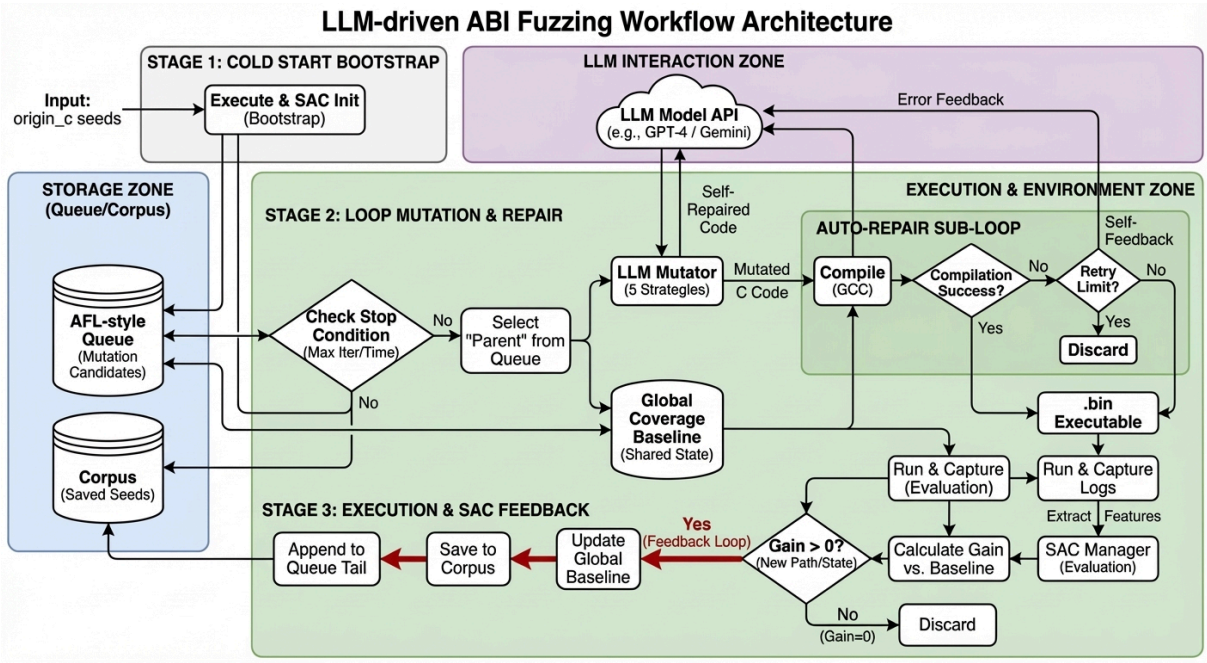

Figure 1 Overall pipeline of ABIscope.

2.2 Seed Initialization

Linux ABI Compatibility is complicated by the fact that the Linux kernel provides a large number of system calls and different structures, each of them have their own ABI. It is impractical to generate test cases for all of them. To simplify the problem, we focus on a subset of system calls that are commonly used and have well-defined ABIs. We select a set of system calls of file system, involves `open`, `read`, `write`, `close`, `mmap`, `munmap`, `ioctl` and so on. The detailed definition of these syscalls can be found in the appendix.

Syzkaller is a popular fuzzing tool for finding bugs in the Linux kernel developed by Google. It generates random inputs for system calls and monitors the kernel's behavior to identify potential vulnerabilities. But we cannot directly use Syzkaller to generate test cases for ABI Compatibility testing because incompatibility issues may not necessarily cause crashes or other easily detectable symptoms. That is to say, Syzkaller does not monitor the output of the system calls, which is crucial for identifying ABI Compatibility issues. However, Syzkaller can be used to generate initial seed test cases for our tool. We use syzkaller to generate a large number of test cases for the selected system calls and then filiter out the valid ones based on their output. These valid test cases should be able to execute successfully on the Linux kernel

and produce expected results. After the filitering process, we use a pre-written script to print the syscalls return value and the output parameters in a structured format. These output will be used to evaluate the compatibility of the test cases on Rust-based kernels.

2.3 LLM-based mutation

ABIscope adopts a mutation-based approach to generate new test cases from the initial seeds. We leverage LLMs to perform mutations on the valid test cases generated in the previous stage. The LLM taks the valid test cases as input and generates new test cases by applying various mutation strategies.

- Structure mutation: LLM can insert new syscalls to the test cases which can trigger new code paths in the kernel. For example, LLM can insert`mmap` syscall before `read` syscall to trigger the code path of memory mapping in the kernel. Also, LLM can remove some syscalls from the test cases to reach the edge cases. For example, LLM can remove `open` syscall from the test case to trigger the code path of handling invalid file descriptors to see the ability of the error-handling code in the rust-based kernel.
- Order mutation: LLM can change the order of the syscalls in the tests cases to trigger different code paths in the kernel. For example, LLM can change the order of `rmdir` and `rename` syscalls to trigger the code path of handling directory operations in the kernel.
- Fault injection: LLM can inject faults into the test cases to trigger specific error-handling code paths in the kernel. We can find incompatibility issues through the different errno values returned by the syscalls.
- Logic mutation: LLM can construct stateful misuse patterns like double free, use-after-free and so on. The idea is the same as the logic mutation in AFL++ to see the compatibility of error-handling.
- Parameter mutation: LLM perturbs syscall arguments at both value and structure levels to expose ABI-sensitive boundary behavior. Concretely, it rewrites scalar parameters (e.g., file descriptors, offsets, lengths, and flags) toward the cases that never appear in the seeds before. For instance, it may generate negative offsets, oversized lengths, or uncommon flag

combinations. LLM can effectively explore the argument space by learning from the existing seeds.

2.4 Execution and Validation

2.5 Coverage-Guided Selection

To quantify the effectiveness of LLM-generated mutations, ABIscope adopts SAC (System-call Argument Coverage) as its primary feedback signal. In contrast to conventional control-flow coverage (e.g., line, basic-block, or branch coverage collected through kernel instrumentation), SAC evaluates whether a mutated seed explores previously unseen argument semantics and externally observable execution outcomes. This formulation is particularly suitable for ABI compatibility analysis, where behavioral divergence may arise even when no crash or obvious control-flow anomaly is observed.

In our implementation, SAC integrates both static and dynamic evidence. The static dimension is extracted directly from generated C programs and includes: (i) flag-level behaviors, where individual macros (such as `O_RDWR`, `O_CREAT`, and `SEEK_END`) and their co-occurrence relations are encoded; (ii) path-pattern abstractions, where path-like literals are mapped to semantic categories defined by preconfigured pattern rules; (iii) numeric and alignment-sensitive arguments, including boundary values (e.g., `-1`, `0`), page-aligned constants, and oversized unsigned probes; and (iv) permission-mask semantics, where mode-related expressions are matched against predefined privilege-related masks to detect bits such as SUID/SGID.

The dynamic dimension is obtained from runtime logs after compilation and execution of each seed. Through syscall-result probes (e.g., `LOG_SYSCALL_RES`), the framework records ordered syscall occurrences, per-call success/failure status, concrete errno outcomes, and global errno visibility. In addition, side effects on filesystem objects are characterized through `stat`-derived attributes, including file size regimes, link-count transitions, and effective mode changes. These dynamic observations provide direct evidence of kernel-visible state transitions and therefore complement static argument analysis.

All extracted signals are normalized into a unified feature set. For a new seed, ABIscope computes coverage gain as the cardinality of the set difference between the seed feature set and the accumulated global coverage set (equivalently, `gain = len(new_features - global_coverage)`). Seeds with positive gain are admitted into the corpus and prioritized for subsequent mutation rounds, while seeds with zero gain are typically discarded. Consequently, SAC serves as a non-instrumentation, semantics-oriented guidance mechanism that continuously steers the LLM toward high-value edge cases for ABI incompatibility discovery.

Also, the mutation strategies are not applied randomly. Instead, we use a reinforcement learning-based approach to select the mutation strategy for each seed. The reward signal is based on the coverage gain of the mutated seed. The mutation strategy that leads to higher coverage gain will be selected more frequently in the future. This approach allows ABIscope to adaptively focus on the most promising mutation strategies and improve the efficiency of selection process. The overall pipeline will be repeated until it reaches the maximum number of iterations or the time budget is exhausted.

2.6 Threats to Validity

Despite the feedback-driven design of ABIscope, several validity threats still remain.

First, there are construct-validity threats in the coverage signal. SAC is designed to capture argument semantics and observable side effects, but it is still a proxy metric rather than a direct measure of incompatibility severity. A seed may produce positive SAC gain without exposing a meaningful ABI defect, while some true incompatibilities may exhibit limited gain if their behavioral signatures overlap with already observed patterns. To reduce this risk, SAC is evaluated jointly with differential execution outcomes and error-code distributions, rather than used as a standalone criterion.

Second, there are internal-validity threats in mutation and execution. LLM-based mutation is inherently stochastic and can be sensitive to prompt context, which may affect reproducibility across runs. In addition, generated tests can contain invalid or weakly constrained arguments; such cases may amplify input noise and obscure real kernel-level divergence. We therefore retain

only compilable/runnable candidates and interpret failures together with runtime traces, so that parameter invalidity can be separated from compatibility-relevant behavioral differences.

Third, differential execution itself introduces confounding factors. Linux and Rust-based kernels may differ in scheduling behavior, timing, default configuration, and tolerance to undefined or implementation-dependent behavior. Some output discrepancies can therefore reflect environmental or semantic ambiguity rather than strict ABI incompatibility. To mitigate this issue, ABIscope compares aligned test inputs under controlled conditions and emphasizes stable, repeatable divergences.

Finally, there are external-validity limitations. The current setup focuses on a selected file system-related syscall subset and may not fully generalize to other subsystems (e.g., networking, IPC, device drivers) or production workloads. Consequently, conclusions should be interpreted as evidence for the studied syscall domain, with broader generalization requiring expanded syscall coverage and larger-scale cross-kernel evaluation.

3. Initial Experimental Results

This chapter focuses on differential results of file-system-related syscalls (e.g., `openat`, `read`, `write`, `fcntl`, `renameat`).

3.1 KeepOS

- In a subset of cases, KeepOS and Linux show errno mismatches (e.g., different paths leading to `EINVAL` or `EBADF`).
- There are behavioral divergences for specific flag combinations.
- Some cases exhibit “Linux fails but KeepOS succeeds” (or the reverse), suggesting syscall semantics are not fully aligned yet.

Corresponding logs can be found in `abi/origin_c/run_logs_keepos/`.

3.2 Asterinas

- Asterinas, as another Rust-based kernel target, is used for horizontal comparison of incompatibility categories.

- For the file-system module, we focus on return-value and errno consistency of open-family and metadata-related calls.
- In the next stage, the same sample set will be executed on Asterinas to produce side-by-side statistics.

4. Discussion

讨论重点包括：

- ABI 差异的可复现性：同一程序多次运行是否稳定；
- 不兼容类型聚类：是集中在参数校验、权限检查，还是文件状态机逻辑；
- 工程可修复性：是否可通过补齐 syscall 语义或调整 errno 映射快速收敛。

同时，LLM 分析用于缩短定位时间，但最终结论仍需要内核实现与测试用例双重验证。

5. Summary and Future Work

本文构建了面向 Rust-based kernel 的 ABI 差分分析流程，形成了“样本生成—双平台执行—日志对齐—LLM 归因”的闭环。

未来工作包括：

- 扩展到更多 syscall 家族（网络、进程管理、内存映射）；
- 建立不兼容严重度评分与修复优先级机制；
- 引入回归测试集，将已修复案例长期纳入 CI。

References

Appendix

Appendix A Code Example

```
#include <iostream>
using namespace std;

int main() {
  cout << "Hello, world!" << endl;
  return 0;
}
```